

React — Day 1

Components, JSX, Props, Lists & Keys

Zero to interactive UI in one session

Prerequisites: working knowledge of JavaScript (ES6)

Environment: Node.js 18+, VS Code, Chrome

1. JavaScript warm-up

More people struggle with React because of JavaScript than because of React itself.

These five features appear in every React file you will ever open. Read each line and predict the result before running it. If any of them are unfamiliar, that is the thing to fix today — not React.

```
// 1. Destructuring – this is exactly what props look like
const user = { name: "Sara", age: 24 };
const { name, age } = user;

// 2. map() – this is how every list in React is rendered
const nums = [1, 2, 3];
const doubled = nums.map(n => n * 2);          // [2, 4, 6]

// 3. Spread – this is how you update state without mutating
const updated = { ...user, age: 25 };          // new object
const moreNums = [...nums, 4];                 // new array

// 4. Template literals
const greeting = `Hello ${name}, you are ${age}`;

// 5. Optional chaining and short-circuit
const city = user.address?.city;                // undefined, not a crash
const label = name && `Welcome ${name}`;
```

Where each one shows up

JavaScript feature	Where you will meet it in React
Destructuring	Reading props: <code>function Card({ name, price })</code>
<code>map()</code>	Rendering any list of items on screen
Spread	Updating state without mutating it (covered later)
Template literals	Building class names and display strings
Arrow functions	Event handlers: <code>onClick={() => ...}</code>
Optional chaining	Reading data that has not arrived yet (covered later)
Ternary and <code>&&</code>	Showing or hiding parts of the UI
ES modules	Every component lives in its own file

2. What React is

React is an open-source JavaScript library for building user interfaces. It is a *library*, not a framework — it handles the view layer and leaves routing, data fetching, and state management to you or to other packages. Those surrounding pieces are covered later in the course.

Where it came from

Year	Milestone
2011	Created by Jordan Walke , a software engineer at Facebook. First deployed on the Facebook news feed.
2012	Adopted by Instagram after Facebook acquired it.
2013	Open-sourced at JSConf US in May. The reaction was initially hostile — mixing markup into JavaScript was considered bad practice at the time.
2015	React Native brings the same model to iOS and Android.
2019	Hooks arrive in version 16.8. Function components can now hold state, and class components effectively become legacy. Everything in this course uses hooks.
2024	React 19 released in December, adding actions, form handling, and the <code>use</code> API.
2025	React Compiler reaches version 1.0 in October, automating performance optimisation.
2026	The React Foundation launches under the Linux Foundation in February. React, React Native, and JSX move from Meta's ownership to independent governance.

Versions

Question	Answer
Current release	19.2.8 , published July 2026. The current minor line is 19.2.
What you will install	<code>npm create vite@latest</code> gives you the latest 19.x automatically. You do not need to pin a version.
Is there an LTS version?	No. React does not have long-term support releases the way Node.js does.
What is supported, then?	React follows semantic versioning. The latest major gets new features and bug fixes; earlier majors receive critical security fixes only.
Which version do tutorials assume?	Anything written before 2024 will likely predate React 19, and anything before 2019 predates hooks entirely. Check the date before trusting a tutorial.

Practical note:

because there is no LTS, "which version should we use" has a simple answer in React — the latest stable one. Upgrades within a major version are routine, and React's team is unusually careful about breaking changes.

The one idea that holds everything together

Before any syntax, understand this sentence, because every topic in this course is a footnote to it:

UI is a function of state.

You describe what the screen should look like for a given set of data. React figures out which parts of the DOM to change. You never write the update steps yourself.

In plain JavaScript you write *instructions*: find this element, change its text, add this class, remove that node. In React you write the *result*: given this data, here is what the page looks like. React handles the difference between the old screen and the new one.

3. Why React exists

The same counter, written twice.

Vanilla JavaScript

```
<button id="btn">Clicked 0 times</button>

<script>
  let count = 0;
  const btn = document.getElementById("btn");

  btn.addEventListener("click", () => {
    count++;
    btn.textContent = `Clicked ${count} times`; // manual DOM update
  });
</script>
```

Now imagine the requirements grow: show the same count in a second place on the page, add a reset button, and disable the button after ten clicks. Every one of those additions means another line that manually pokes at the DOM. The number of update instructions grows faster than the number of features.

React

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

There is no `textContent` assignment. You state that the button shows `count`, and that clicking increases `count`. Showing the value in a second place is one more line of JSX and zero extra update logic, because there is no update logic anywhere.

4. Project setup

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

Do not use Create React App.

It is deprecated and no longer maintained. If a tutorial you find online starts with `npx create-react-app`, it is old enough that other things in it will also be out of date. Vite is the standard for new projects.

The three files that matter

Read them in this order, because it follows the actual flow of control.

File	What it does
<code>index.html</code>	The single real HTML file in the project. It contains one meaningful line: <code><div id="root"></div></code> . Everything else on the page is created by JavaScript. This is what "single page application" means.
<code>src/main.jsx</code>	The entry point. <code>createRoot(document.getElementById("root")).render(<App />)</code> is the bridge from the real DOM into React. It runs exactly once, when the app starts.
<code>src/App.jsx</code>	Your first component, and the root of your component tree.

Before writing anything, delete the Vite boilerplate: clear out `App.css` and strip `App.jsx` down to a single `<h1>`. The default template ships with spinning logos and state already wired up, and it is genuinely hard to tell which parts are React and which are decoration.

Imports and exports

Every component lives in its own file, so you need to move them between files. There are two styles and they behave differently.

```
// ProductCard.jsx – default export, one per file
export default function ProductCard() { ... }

// Importing it: the name is yours to choose
import ProductCard from "../components/ProductCard";
import Anything      from "../components/ProductCard"; // also works
```

```
// helpers.js – named exports, as many as you like
export function formatPrice(n) { ... }
export const TAX_RATE = 0.14;

// Importing them: names must match exactly
import { formatPrice, TAX_RATE } from "../helpers";
```

The convention:

one component per file, exported as the default, with the filename matching the component name. Named exports are for utilities and constants. Mixing both in one file is legal but rarely worth it.

Note that `../` is required for your own files. `import x from "helpers"` looks in `node_modules` and will fail; `import x from "../helpers"` looks in the current folder.

5. React DevTools

Install the React Developer Tools extension for Chrome or Firefox before going any further. It is the fastest way to make the component tree feel real rather than theoretical, and it is a debugging tool you will use every single day.

Once installed, two new tabs appear in your browser's developer tools.

Tab	What it shows
Components	Your component tree, exactly as you wrote it — <code>App</code> containing <code>Panel</code> containing six <code>ProductCard</code> s. Select any one to see its props and state in the side panel.
Profiler	Which components re-rendered and how long each took. You will not need this today, but it is the tool that answers "why is this slow" later on.

Three things to try right now

1. **Walk your tree.** Find your six `ProductCard` instances. Notice that the panel shows the props each one received — this is the clearest possible picture of data flowing downward.
2. **Edit a prop live.** Select a card, change its `name` value directly in the DevTools panel, and watch the page update instantly. Nothing was recompiled and no file was saved.
3. **Find the DOM node.** Use the eye icon to jump from a React component to the actual DOM element it produced, so you can see the relationship between the two.

Why the second exercise matters.

Editing a prop and watching the screen change is the most direct demonstration of *UI is a function of state* that exists. You changed the data; you did not touch the DOM; the screen updated anyway. Keep that image in mind for the rest of the course.

If the Components tab is missing or greyed out,

you are probably looking at a page that does not use React, or the dev server is not running. It only activates on pages where React is present.

6. JSX

What JSX actually is

JSX is not HTML, and it is not a template language. It is shorthand for a function call. This:

```
<h1 className="title">Hello</h1>
```

compiles to roughly this:

```
_jsx("h1", { className: "title", children: "Hello" })
```

The result is an ordinary JavaScript object describing what should appear on screen. It is not a DOM node, and nothing has been rendered yet. React reads these objects and decides what to do with the real DOM.

Every JSX rule below follows from this fact.

Once you accept that you are writing function calls in disguise, the rules stop being arbitrary and become obvious.

The rules

One root element per return

A function can only return one value. If you need siblings, wrap them — either in a real element or in a fragment, which produces no DOM node at all.

```
// Error: two things returned
return <h1>Title</h1><p>Text</p>;

// Fine
return <div><h1>Title</h1><p>Text</p></div>;

// Better when you do not want an extra div in the DOM
return <><h1>Title</h1><p>Text</p></>;
```

Curly braces escape into JavaScript

Anything inside `{ }` is evaluated as a JavaScript expression. Any expression works — a variable, arithmetic, a function call, a ternary. Statements such as `if` and `for` do not, because they produce no value.

```
<p>{user.name}</p>
<p>{2 + 2}</p>
<p>{formatPrice(product.price)}</p>
<p>{items.length > 0 ? "Has items" : "Empty"}</p>
```

Attribute names follow the DOM, not HTML

JSX props map to **DOM properties**, not to HTML attributes. That single fact explains every renaming you will meet. Open a console on any page and check for yourself:

```
document.querySelector("div").class      // undefined
document.querySelector("div").className // "card"
document.querySelector("label").htmlFor // the label's "for" value
```

The HTML *attribute* is `class`, but the JavaScript *property* for reading and writing it has always been `className`. That naming is part of the DOM specification and predates React by about fifteen years. React did not invent the rename — it inherited it.

HTML attribute	JSX prop
<code>class</code>	<code>className</code>
<code>for</code>	<code>htmlFor</code>
<code>tabindex</code>	<code>tabIndex</code>
<code>readonly</code>	<code>readOnly</code>
<code>maxlength</code>	<code>maxLength</code>
<code>colspan</code> / <code>rowspan</code>	<code>colSpan</code> / <code>rowSpan</code>
<code>onclick</code>	<code>onClick</code>

Once you see that JSX props are JavaScript properties rather than HTML attributes, this stops being a list of exceptions and becomes one rule. Event handlers are camelCase for exactly the same reason: `onClick`, `onChange`, `onSubmit`.

A correction to something you will read elsewhere.

Most tutorials say `class` was renamed because it is a reserved word in JavaScript. That is a tidy story, but it is not accurate — reserved words have been legal as object keys since ES5, so `{ class: "card" }` is perfectly valid JavaScript today. The real reasons are historical and consistency: in 2013 React still had to run on older engines where this was a genuine problem, and the team chose to mirror DOM property names throughout the API. The DOM-property explanation is worth learning because it also tells you why `for`, `tabindex`, and `onclick` change. The reserved-word story explains one case and then stops working.

What happens if you write `class` anyway

React warns you in the console:

```
Warning: Invalid DOM property `class`. Did you mean `className`?
```

But — and this surprises people — in current React the `class` **is still applied**. Both of these produce identical HTML:

```
<div class="card">Hi</div>      → <div class="card">Hi</div>
<div className="card">Hi</div>  → <div class="card">Hi</div>
```

React passes the unrecognised prop straight through to the attribute rather than discarding it. The same is true of `for` versus `htmlFor`.

That it works is exactly why it is a trap.

You get a console warning on every render, your linter flags it, and it quietly breaks the mental model that props map to DOM properties — which is the thing that tells you why `htmlFor` and `tabIndex` look the way they do. Note also that older versions of React discarded the prop entirely, which is why you will find contradictory advice online. Use `className`.

Every tag must close

Self-closing tags need the slash: `<img />`, `<br />`, `<input />`. HTML forgives you here; JSX does not.

Inline styles are objects

```
<div style={{ color: "red", fontSize: 16, marginTop: 8 }}>
```

The outer brace enters JavaScript; the inner brace is the object literal. Property names are camelCase because they are JavaScript keys, and numbers default to pixels.

Comments

```
{/* This is a JSX comment */}
```

Error you will definitely hit: "Objects are not valid as a React child."

This means you put an object where text was expected — usually `{user}` instead of `{user.name}`. React can render strings, numbers, and elements. It cannot render a plain object, because it has no idea how you want it displayed.

7. Components and props

A component is a function that returns JSX

```
function Greeting() {
  return <h2>Hello there</h2>;
}

function App() {
  return (
    <div>
      <Greeting />
      <Greeting />
      <Greeting />
    </div>
  );
}
```

The capital letter is not a style convention — it is load-bearing.

JSX uses capitalisation to decide whether a tag is one of your components or a plain DOM element. `<Greeting />` calls your function. `<greeting />` is treated as an unknown HTML tag: it renders nothing and gives you no useful error. This mistake can cost twenty minutes if you do not know to look for it.

Props are function arguments

```
function Greeting({ name, role }) {
  return <h2>Hello {name}, you are a {role}</h2>;
}

<Greeting name="Sara" role="developer" />
```

Strings are passed with quotes. Everything else — numbers, booleans, arrays, objects, functions — is passed with braces:

```
<ProductCard
  name="Laptop"           // string
  price={999}             // number
  inStock={true}          // boolean
  inStock                 // shorthand: same as {true}
  tags={['new', 'sale']}  // array
  onBuy={handleBuy}       // function
/>
```

Default values go in the destructuring:

```
function ProductCard({ title = "Untitled", price = 0 }) { ... }
```

Props are read-only, and data flows one way.

Assigning to a prop inside a component does not work. A parent passes data down to a child; a child never reaches up and modifies what it was given. This one-directional flow is what makes React apps predictable — when a value is wrong, you only have to look in one place.

Composition with children

Anything placed between a component's opening and closing tags arrives as the `children` prop. This is the most useful pattern in React and the one most beginner tutorials skip.

```
function Panel({ title, children }) {
  return (
    <section className="panel">
      <h3>{title}</h3>
      {children}
    </section>
  );
}

<Panel title="Settings">
  <p>Anything at all can go in here.</p>
  <button>Save</button>
</Panel>
```

Without `children`, a reusable card or layout would need a prop for every possible piece of content it might hold. With it, the component owns the wrapper and the caller owns the contents. Layouts, modals, cards, and page shells are all built this way.

Computing className

`className` is just a string, and you are in JavaScript, so you can build it however you like. This comes up constantly — highlighting a selected item, fading a disabled one, colouring a status badge.

```
// Template literal – good when there is a base class plus a modifier
<div className={`card ${inStock ? "" : "card--faded"}`>

// Ternary – good when the two states are unrelated
<span className={inStock ? "badge-green" : "badge-red"}>

// Built up in advance – good when the logic gets long
const classes = ["card"];
if (!inStock) classes.push("card--faded");
if (price > 500) classes.push("card--featured");

<div className={classes.join(" ")}>
```

Watch the stray space.

`{`card ${x && "extra"}`}` produces `"card false"` when `x` is false, because the falsy value gets stringified into the class list. Use a ternary with an empty string instead, as in the first example above.

8. Rendering lists

```
const products = [
  { id: 1, name: "Laptop", price: 999 },
  { id: 2, name: "Phone", price: 599 },
  { id: 3, name: "Tablet", price: 449 },
];

function ProductList() {
  return (
    <ul>
      {products.map(p => (
        <ProductCard key={p.id} name={p.name} price={p.price} />
      ))}
    </ul>
  );
}
```

Two things are happening. The `.map()` sits inside `{ }` because it is JavaScript, and it returns an array of elements — React knows how to render an array of children.

Keys, and why the wrong one breaks your app

When state changes, React re-runs your component and produces a fresh tree of element objects. It then compares that tree against the previous one to work out the smallest set of DOM changes needed. This comparison is called **reconciliation**, and for every item in a list it must answer one question:

Is this new element the same element as one from the previous render, or a different one?

That answer decides everything:

- **"Same element"** — React keeps the existing DOM node alive and patches only the attributes that changed.
- **"Different element"** — React destroys the old DOM node and builds a new one from scratch.

Keeping a node alive matters because a live DOM node carries state that is not in your array: text typed into an `<input>`, checkbox state, keyboard focus, scroll position, and any `useState` inside a child component. React is not tracking any of that. It survives only because the node itself survived.

The key is how React answers the question. With no key, React falls back to matching by position in the list.

Watching it break

Three items rendered with `key={index}`. Each row has an input the user has typed into.

Key	Item	Typed into input
0	Apples	"red ones"
1	Bread	"sourdough"
2	Cheese	"cheddar"

Now delete **Apples**. The array becomes `[Bread, Cheese]`, and React re-renders with keys 0 and 1. Here is the diff React performs:

Key	Before	After	React's decision
0	Apples	Bread	Key still exists → same element. Reuse the node, patch the label. The input is untouched and still holds "red ones".
1	Bread	Cheese	Key still exists → same element. Reuse, patch the label. Input still holds "sourdough".
2	Cheese	—	Key gone → unmount. The node is destroyed, and "cheddar" is destroyed with it.

Result on screen — wrong

Bread	"red ones" (this was Apples' note)
Cheese	"sourdough" (this was Bread's note)

Every note moved up one row, and the last one vanished. Nothing malfunctioned. You told React that the item at position 0 is still there and merely changed its name, so React updated the label and left the rest of the node alone. As far as React could tell, the item that disappeared was the *last* one, because that is the position that stopped existing.

The fix

Now the same deletion with `key={item.id}`, where the ids are `a`, `b`, `c`:

Key	Before	After	React's decision
a	Apples	—	Key gone → unmount. "red ones" is destroyed, which is correct, because Apples is what was deleted.
b	Bread	Bread	Found by key → reuse the whole node. The label has not even changed, so nothing is patched.
c	Cheese	Cheese	Found by key → reuse. "cheddar" stays where it belongs.

Result on screen — correct

Bread	"sourdough"
Cheese	"cheddar"

The rules

Keys must be unique among siblings, not globally

Two different lists on the same page can both use key `1`. React only compares keys within a single array of children.

The key goes on the outermost element inside the map

```
// Correct
{products.map(p => <ProductCard key={p.id} product={p} />)}

// Does nothing – too late, the array was already built
function ProductCard({ product }) {
  return <div key={product.id}>...</div>;
}
```

key is not a prop

Inside the component, `props.key` is `undefined`. React strips it out for its own bookkeeping. If the component needs the id as well, pass it twice:

```
<ProductCard key={p.id} id={p.id} />
```

Math.random() is worse than the index, not better

A fresh key on every render means React concludes that every item is brand new. It destroys and rebuilds the entire list every time — losing all state, dropping focus mid-typing, and destroying performance.

When the index is acceptable

Only when all of the following are true: the list never reorders or sorts, items are only added or removed at the end, and the rows contain no inputs, focus, or state. If any one of those could change, use a stable id.

Bonus: changing a key on purpose resets a component.

Because a new key forces React to throw away the old node and mount a fresh one, you can use this deliberately:

```
<ProfileForm key={userId} user={user} />
```

When `userId` changes, React unmounts the old form entirely and mounts a new one with clean state — so one user's half-typed edits never leak into the next user's form. We will come back to this once mounting has been covered.

9. Conditional rendering

```
// Ternary – when you need both branches
{isLoggedIn ? <Dashboard /> : <LoginForm />}

// && – when you only need one
{hasError && <p className="error">Something went wrong</p>}

// Early return – when the whole component differs
function Profile({ user }) {
  if (!user) return <p>Loading...</p>;
  return <h1>{user.name}</h1>;
}
```

The trap: why a stray 0 appears on screen

```
{items.length && <p>You have items</p>} // renders "0" when the list is empty
```

The mistake is thinking `&&` is a conditional. It is not. In JavaScript, `&&` is an operator that **returns one of its two operands**, never a boolean:

- `a && b` evaluates `a`. If `a` is falsy, it returns `a` **itself** and never looks at `b`.
- If `a` is truthy, it returns `b`.

So with an empty array, `items.length` is `0`, which is falsy, so the whole expression short-circuits and returns the **number zero** — not `false`. React is now being handed the number 0 and asked to render it.

The second half of the puzzle is React's rules for what each value renders as:

Value inside { }	What appears	Safe as a guard?
"hello"	hello	—
42	42	—
0	0	No — this is the bug
NaN	NaN	No — the sneakier version
true / false	nothing	Yes
null / undefined	nothing	Yes
""	nothing	Yes, by luck

React skips booleans, `null`, and `undefined` deliberately — that is precisely what makes the `condition && <Element />` pattern work at all. But it *must* render numbers, because `<p>{count}</p>` with a count of zero has to display "0". React has no way to know your `0` was meant as a guard rather than as data.

So the bug is not really about `&&` or about React on its own. It is the collision of two individually reasonable rules: JavaScript returns the falsy operand, and React prints numbers.

The fix: guarantee the left side is a boolean.

```
{items.length > 0 && <p>You have items</p>} // explicit – best  
{!!items.length && <p>You have items</p>} // double negation  
{items.length ? <p>You have items</p> : null} // ternary
```

Rule to memorise:

the left side of `&&` in JSX must always be a boolean expression, never a raw number. Watch for `.length`, counts, prices, quantities, and IDs. And watch for `NaN` — `{total * qty && <p>Ready</p>}` will print the literal text "NaN" on your page if either value is undefined, which looks like a rendering engine failure rather than a typo.

10. Introduction to hooks

Everything so far has been static. A component received props and returned JSX, and the only way anything on screen changed was if you edited the code. Hooks are what give components memory and behaviour.

What a hook is

A hook is a function that lets a component tap into a React feature.

State, lifecycle, context, refs — these all live inside React, and a hook is the doorway to one of them from inside your function.

They are ordinary JavaScript functions. What makes them special is that React keeps track of every call, so the value one returns can persist between renders rather than being rebuilt from scratch each time.

Why they exist

Before 2019, only class components could hold state. A component that needed to remember anything had to be rewritten as a class, with `this.state`, `this.setState`, constructors, and manual method binding. Logic that you wanted to reuse across components was awkward to extract, and lifecycle methods forced unrelated concerns into the same place.

Hooks arrived in React 16.8 and removed all of that. Function components could now do everything classes could, with far less ceremony. Classes still work and you will meet them in older code, but no new React is written that way. Everything in this course uses hooks.

How to recognise one

Every hook is named `use` followed by a capital letter. This is not decoration — React's linting tools rely on the prefix to check that you are using them correctly, and you will follow the same convention when you write your own.

Hook	What it gives the component
<code>useState</code>	A value that survives between renders, plus a way to change it. Today's hook.
<code>useEffect</code>	A way to synchronise with something outside React — a network request, a subscription, a timer. Covered later.
<code>useRef</code>	A reference to a DOM node, or a value that persists without triggering re-renders. Covered later.
<code>useContext</code>	Access to data shared across many components without passing props down manually. Covered later.
<code>useMemo</code> / <code>useCallback</code>	Performance tools for expensive work. Covered later.

You can also write your own — a **custom hook** is just a function starting with `use` that calls other hooks. That is how logic gets shared between components, and it is covered later in the course.

Two rules

1. Only call hooks at the **top level** of a component — never inside a condition, a loop, or a nested function.
2. Only call hooks from React function components, or from your own custom hooks. Not from ordinary functions.

```
// Broken – the hook is inside a condition
function Profile({ isLoggedIn }) {
  if (isLoggedIn) {
    const [name, setName] = useState("");
  }
}

// Correct – the hook always runs; the condition is applied after
function Profile({ isLoggedIn }) {
  const [name, setName] = useState("");
  if (!isLoggedIn) return <p>Please log in</p>;
}
```

The reason, in one sentence:

React does not know your variable names — it tracks state by the *order* the hooks were called in. Put one inside an `if` and the order shifts between renders, so React hands state belonging to one variable to a different one. We will go into this properly in the next session; for now, keep every hook at the top of the component and you will be fine.

11. useState

The first and most important hook. It gives a component a value that survives between renders.

Anatomy

```
import { useState } from "react";
//    ^ named export, so the braces are required

const [count, setCount] = useState(0);
//    ^           ^           ^
//    |           |           | the INITIAL value, used only
//    |           |           | on the very first render
//    |           |           | the function that changes it
//    |           |           | the current value for this render
```

`useState` returns an array of exactly two things, and we destructure it on the spot. That is why Day 1 opens with destructuring practice. The names are yours to choose — `[count, setCount]` is convention, not a requirement, though following it makes your code readable to everyone else.

A working counter

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

What actually happens when you click

1. The click handler runs and calls `setCount`.
2. React records the new value and marks this component as needing a re-render.
3. React runs the `Counter` function again from the top.
4. This time `useState` returns the *new* value rather than the initial one.
5. The function returns fresh JSX, React compares it with the previous version, and updates only what changed.

Note step 3: the whole function body runs again on every render. Putting a `console.log` at the top of a component is therefore a very effective way to see how often it is running.

Three things to hold onto

- The initial value is used **only once**, on the first render. Passing `useState(0)` does not reset the count to zero on every render.
- Calling the setter is what tells React something changed. Without it, React has no idea.
- You **never** assign to the value directly. `count = 5` does nothing at all — no error, no re-render, no change on screen.

A component can hold as many pieces of state as it needs

```
const [name, setName] = useState("");
const [age, setAge] = useState(0);
const [tags, setTags] = useState([]);
const [user, setUser] = useState(null);
```

Any type of value works — strings, numbers, booleans, arrays, objects, `null`. Group values into one object when they always change together; keep them separate when they change independently.

A controlled input

```
function NameInput() {
  const [name, setName] = useState("");

  return (
    <>
      <input value={name} onChange={e => setName(e.target.value)} />
      <p>Hello {name || "stranger"}</p>
    </>
  );
}
```

Two halves make this work. `value={name}` means the input displays whatever is in state. `onChange` means every keystroke writes back to state. Remove either half and the input stops working.

Try deleting the `onChange` handler, then type in the box.

Nothing happens. The input is frozen. That is the proof that the JSX is genuinely driven by state rather than the other way around — the input cannot change unless the state changes first.

One mistake worth seeing now.

Write `onClick={setCount(count + 1)}` and the app crashes with "Too many re-renders". That form *calls* the setter immediately during render, which schedules a render, which calls it again. You want `onClick={() => setCount(count + 1)}` — a function React can call *later*, when the click actually happens.

State has more depth to it than this — how updates are batched, why you must never modify state directly, and how components share state between them. That is the focus of the next session. For today, the goal is simply that a component can remember something and update the screen when it changes.

12. Spot the bug

Each snippet below is broken in exactly one way, and every one of these is a mistake you will make for real within your first week. Find the fault and write the fix. Recognising the symptom is what saves you time later — not memorising the rule.

1. Nothing appears on the page, and there is no error

```
function greeting() {  
  return <h2>Hello</h2>;  
}  
  
export default function App() {  
  return <div><greeting /></div>;  
}
```

2. The styling never applies

```
<div class="card">  
  <h3>Laptop</h3>  
</div>
```

3. "Objects are not valid as a React child"

```
const user = { name: "Sara", age: 24 };  
  
return <p>Welcome back, {user}</p>;
```

4. A warning in the console about a missing prop

```
{products.map(product => (  
  <li>{product.name}</li>  
))}
```

5. A stray "0" appears above the list when it is empty

```
{cart.length && <p>You have {cart.length} items</p>}
```

6. The page shows the text "price" instead of a number

```
function ProductCard({ price }) {  
  return <p>Costs $price</p>;  
}
```

7. Typing in the box does nothing

```
const [name, setName] = useState("");  
  
return <input value={name} />;
```

Answers

are in the error table in the quick reference at the back. Try all seven before turning to it.

Task

Build a product catalogue using what we covered today: components, props, `children`, lists and keys, conditional rendering, and one piece of state.

```
const initialProducts = [
  { id: 1, name: "Laptop",    price: 999, category: "Computers",  inStock: true },
  { id: 2, name: "Phone",    price: 599, category: "Mobile",    inStock: true },
  { id: 3, name: "Tablet",   price: 449, category: "Mobile",    inStock: false },
  { id: 4, name: "Monitor",  price: 329, category: "Computers",  inStock: true },
  { id: 5, name: "Keyboard", price: 89,  category: "Accessories", inStock: false },
  { id: 6, name: "Mouse",    price: 45,  category: "Accessories", inStock: true },
];
```

Requirements

1. Build a `ProductCard` component that receives its data through props and displays the name, price, and category. Render all six with `.map()`, giving each a key taken from the product id.
2. Build a reusable `Panel` component taking a `title` prop and `children`, and wrap the grid in it.
3. Hold the product list in one piece of state — `const [products, setProducts] = useState(initialProducts);` — and make it the only `useState` in the task.
4. Show an "Out of stock" badge only where `inStock` is false. Use the `&&` pattern — carefully.
5. Give each card a **Delete** button that removes that product from the list.

Quick reference

Errors you will see today

Message or symptom	What it actually means
Adjacent JSX elements must be wrapped	You returned two sibling elements. Wrap them in a <code><div></code> or a fragment <code><></></code> .
Objects are not valid as a React child	You wrote <code>{user}</code> where you meant <code>{user.name}</code> .
Each child in a list should have a unique "key" prop	You forgot the key on a <code>.map()</code> , or put it on the wrong element.
The component renders nothing, and there is no error	You used a lowercase component name. <code><greeting /></code> is treated as an unknown HTML tag.
A stray "0" appears on the page	<code>{items.length && ...}</code> . Use <code>{items.length > 0 && ...}</code> .
Typing in an input does nothing	You set <code>value</code> without an <code>onChange</code> handler.
Your CSS class has no effect	You wrote <code>class</code> instead of <code>className</code> .
Too many re-renders	You called a setter during render. Use <code>onClick={() => setX(1)}</code> , not <code>onClick={setX(1)}</code> .
Rendered more hooks than during the previous render	A hook is inside a condition or loop. Move it to the top level.
A variable name shows as literal text	You forgot the braces. <code>\$price</code> needs to be <code>{price}</code> .
Notes or inputs attach to the wrong list rows	You used the array index as the key on a list that changes.

Syntax at a glance

Pattern	Syntax
Component	<code>function Name() { return <div />; }</code>
Props in	<code>function Name({ a, b = "default" })</code>
Props out	<code><Name a="text" b={value} c /></code>
Children	<code><Panel>content</Panel></code> → <code>{children}</code>
Expression	<code>{value}</code>
List	<code>{items.map(i => <Row key={i.id} />)}</code>
Conditional, one branch	<code>{ok && <p />}</code>
Conditional, two branches	<code>{ok ? <A /> : }</code>
Fragment	<code><> ... </></code>
Inline style	<code>style={{ color: "red" }}</code>
CSS class	<code>className="card"</code>
Comment	<code>{/* text */}</code>
Import a hook	<code>import { useState } from "react";</code>
Declare state	<code>const [v, setV] = useState(initial);</code>
Update state	<code>setV(newValue)</code>
Event handler	<code>onClick={() => setV(1)}</code> not <code>onClick={setV(1)}</code>